0202. Happy Number

Easy · Math & Geometry

leetcode.com/problems/happy-number

Problem

- Repeatedly replace n with the sum of the squares of its digits.
  n is happy if this reaches 1; otherwise it loops forever.

Intuition

- The sequence either reaches 1 or enters a cycle. Detect the cycle with Floyd's slow/fast pointers (or a HashSet).

Brute Force

- A HashSet of seen values uses extra space.

Optimal Approach

- Slow = n, fast = next(n).
- While fast != 1 and slow != fast: slow = next(slow); fast = next(next(fast)).
- Return fast == 1.

Worked Example

```
n = 19.

19  -> 1 + 81  = 82
82  -> 64 + 4  = 68
68  -> 36 + 64 = 100
100 -> 1   -> happy
```

Java Solution

```java
public class HappyNumber {
    public boolean isHappy(int n) {
        int slow = n, fast = next(n);
        while (fast != 1 && slow != fast) {
            slow = next(slow);
            fast = next(next(fast));
        }
        return fast == 1;
    }

    private int next(int n) {
        int s = 0;
        while (n > 0) { int d = n % 10; s += d * d; n /= 10; }
        return s;
    }
}
```

Complexity

- Time: O(log n) per step, with few steps. Space: O(1).

Pitfalls

- N = 1 is happy immediately.